

软件工程与计算 II 复习提纲含答案版

目录

整理说明	4
第一、二章：软件工程基础	4
题 1：名词解释：软件工程	4
题 2：简答：1950s 到 2000s 软件工程发展的特点	4
第四章：项目启动与项目管理	5
题 3：如何管理团队？在实验中采取哪些办法？有哪些经验？	5
题 4：团队结构有哪几种？	5
题 5：质量保障有哪些措施？结合实验说明	5
题 6：配置管理有哪些活动？实验中如何进行配置管理？	6
题 7：示例：结合实验说明一个项目的质量保障包括哪些活动	6
第五章：需求工程	7
题 8：名词解释：需求	7
题 9：区分需求的三个层次，并给出实例	7
题 10：掌握需求类型，并对实例判定类型	7
第六章：需求分析建模	8
题 11：给定描述时，如何建立用例图、分析类图、系统顺序图和状态图？	8
题 12：图书馆系统：建立用例模型并说明建模过程	8
题 13：ATM 系统：建立领域类图，即分析类图	9
第七章：需求规格说明与测试	10
题 14：为什么需要需求规格说明？结合实验说明	10
题 15：对给定需求规格说明片段，如何判定并修正错误？	10
题 16：对给定需求示例，如何设计功能测试用例？	11
第八章：软件设计基础	11
题 17：名词解释：软件设计	11
题 18：软件设计的核心思想是什么？	11
题 19：软件工程设计有哪三个层次？各层主要思想是什么？	12
第九、十章：体系结构	12

题 20: 体系结构的概念	12
题 21: 体系结构风格的优缺点	12
题 22: 体系结构设计过程	12
题 23: 包的原则	13
题 24: 体系结构构件之间接口的定义	13
题 25: 体系结构开发集成测试用例: Stub 和 Driver	13
题 26: 薪酬管理系统 Web 分层物理包图如何作答?	14
题 27: “社会保险录入”页面的展示层、逻辑层、数据层接口声明	14
第十一章: 人机交互	15
题 28: 名词解释: 可用性	15
题 29: 列出至少 5 个界面设计注意事项并解释	15
题 30: 分析图中两个界面体现或违反的人机交互设计原则	15
题 31: 精神模型、差异性、导航、反馈、协作式设计	16
第十二章: 详细设计	16
题 32: 详细设计的出发点是什么?	16
题 33: 职责分配、协作、控制风格分别是什么?	16
题 34: 给定分析类图、系统顺序图和设计因素, 如何建立设计类图或详细顺序图?	17
题 35: 协作测试中的 MockObject	17
题 36: 示例: 根据 CD 推荐用例画设计类图和集中式控制顺序图	17
第十三章: 耦合、内聚与信息隐藏	18
题 37: 名词解释: 耦合与内聚	18
题 38: 如何说明例子中的耦合程度与内聚, 并给理由?	18
题 39: 信息隐藏的基本思想和两种常见信息隐藏决策	19
第十四章: 模块化原则	19
题 40: Principles from Modularization 九条原则及修正方式	19
第十五章: 封装、OCP、DIP 与 LSP	20
题 41: 信息隐藏与封装的含义	20
题 42: OCP 和 DIP	20
题 43: 替换原则例题: MyStack 继承 Vector 是否合理? Employee 继承 Person 是否合理?	20
第十六章: 设计模式	21
题 44: 如何实现可修改性、可扩展性和灵活性?	21
题 45: 策略模式	21
题 46: 抽象工厂模式	21
题 47: 单件模式	22
题 48: 迭代器模式	22
题 49: 给定场景选择设计模式并写代码, 给出代码要求用设计模式改写	22
第十七、十八章: 软件构造与代码设计	23
题 50: 构造包含哪些活动?	23
题 51: 名词解释: 重构、测试驱动开发、结对编程	23

题 52: 给定代码段, 如何发现问题并改进?	23
题 53: 变量使用、语句处理和 How to write unmaintainable code	24
题 54: 防御与错误处理、契约式设计、防御式编程	24
题 55: 表驱动	24
题 56: 单元测试用例设计	25
第十九章: 测试	25
题 57: 黑盒测试和白盒测试的常见方法, 并比较优缺点	25
题 58: 解释并区别语句覆盖、分支覆盖和路径覆盖	25
题 59: 给定场景, 如何选择测试方法并写测试用例?	26
题 60: JUnit 基本使用方法	26
第二十、二十一章: 维护与演化	27
题 61: 如何理解软件维护的重要性?	27
题 62: 开发可维护软件的方法	27
题 63: 演化式生命周期模型	27
题 64: 用户文档和系统文档	27
题 65: 逆向工程和再工程	28
第二十二、二十三章: 生命周期模型与知识体系	28
题 66: 软件生命周期模型	28
题 67: 解释并比较不同过程模型	28
题 68: 给定场景, 判定适用的开发过程模型	28
题 69: 软件工程知识体系的知识域	29
题型和数量	29
题 70: 考试题型和数量如何理解?	29

整理说明

本文件依据工作区根目录的《复习提纲.pdf》整理。原提纲是 26 页课件式题目清单，没有给出标准答案。本版按原 PDF 的章节顺序补充作答要点、答题模板和典型例题答案。

原 PDF 中出现“结合实验”的题目，但未提供具体实验项目名、团队分工、仓库记录、测试报告和缺陷记录。因此本版给出可直接套用的实验答题模板，不编造具体项目事实；考试或复习时把模板中的项目名称、模块名称、工具链和实际过程替换为自己的实验记录。

第一、二章：软件工程基础

题 1：名词解释：软件工程

答案：

软件工程是把系统化、规范化、可量化的方法应用于软件开发、运行和维护，并研究这些方法的学科。

展开时写三层：

1. 方法层：系统化、规范化、可量化。
2. 对象层：软件制品包括程序、数据、文档和知识，不只是代码。
3. 目标层：在多人协作和长期演化中控制质量、成本、进度和风险，交付可维护的软件系统。

题 2：简答：1950s 到 2000s 软件工程发展的特点

答案：

按时间线作答：

阶段	特点
1950s	商业计算机和早期高级语言出现，软件主要依附硬件，程序规模小，开发依赖个人经验。
1960s	软件规模和复杂度上升，延期、超支、质量低和维护困难集中暴露，1968 年 NATO 会议提出“软件危机”和“软件工程”。
1970s	强调工程化过程，瀑布模型、结构化分析和结构化设计兴起，文档和阶段评审成为控制手段。
1980s	软件生命周期、质量管理、配置管理和成熟度模型发展，团队协作和过程改进成为重点。
1990s	面向对象方法、UML、构件复用和迭代思想普及，软件系统更强调可扩展和可维护。
2000s	敏捷方法、持续集成、开源协作、Web 系统和服务化发展，反馈速度、自动化测试和持续交付成为重要工程能力。

收束句：这条发展线的核心是从“个人写程序”转向“团队用工程方法管理复杂度、变化和质量”。

第四章：项目启动与项目管理

题 3：如何管理团队？在实验中采取哪些办法？有哪些经验？

答案：

团队管理从目标、角色、计划、沟通、质量和风险六方面写：

1. 明确共同目标：把项目目标拆成迭代目标、里程碑和可交付物。
2. 明确角色分工：项目负责人、需求负责人、架构或设计负责人、开发负责人、测试负责人、配置管理员。
3. 制定计划：任务拆分、估算工作量、排定优先级和截止时间。
4. 建立沟通机制：例会、看板、Issue、Pull Request 和评审记录。
5. 控制质量：需求评审、设计评审、代码审查、单元测试、集成测试和回归测试。
6. 管理风险：识别关键成员缺席、需求变更、技术不熟、接口延期等风险，提前准备应对措施。

结合实验的答题模板：

在实验项目中，我们将任务拆为需求、设计、编码、测试和文档五类；通过 Git 仓库、Issue、分支和 Pull Request 跟踪任务；每次迭代结束进行需求和代码评审；对核心功能编写单元测试和集成测试；对延期或接口变化风险设置负责人和截止时间。经验是：分工要落到具体交付物，沟通要有记录，合并代码前要评审和测试。

题 4：团队结构有哪几种？

答案：

可按控制方式和沟通方式分类：

- 主程序员式团队：核心成员负责总体设计和关键决策，适合小团队和技术集中项目。
- 民主式团队：成员平等讨论，适合探索性强、创新性强的任务。
- 分层式团队：项目经理、模块负责人、开发和测试分层协作，适合规模较大的项目。
- 敏捷小队：跨职能成员共同负责迭代交付，强调频繁反馈和持续改进。

答题时说明：团队结构要服务于项目规模、成员能力、需求稳定性和沟通成本。

题 5：质量保障有哪些措施？结合实验说明

答案：

软件质量保证 SQA 关注过程和产品是否满足质量要求。常见措施：

- 需求阶段：需求评审、术语统一、可验证性检查。

- 设计阶段：架构评审、接口评审、设计类图和顺序图检查。
- 构造阶段：代码审查、静态分析、单元测试、持续集成。
- 测试阶段：功能测试、集成测试、回归测试、缺陷跟踪。
- 发布阶段：配置审计、版本核对、冒烟测试和回滚预案。

结合实验的答题模板：

在实验项目中，质量保障贯穿需求、设计、编码、测试和发布。需求文档完成后进行组内评审；设计阶段检查分层结构和接口依赖；编码阶段使用分支开发、Pull Request 和代码审查；核心业务编写单元测试，跨模块流程编写集成测试；每次发布前核对版本、执行冒烟测试并记录缺陷。

题 6：配置管理有哪些活动？实验中如何进行配置管理？

答案：

软件配置管理 SCM 的目标是识别配置项、控制变更、记录状态、审计配置并保证产品与规格一致。

活动包括：

1. 配置项识别：需求文档、设计文档、源代码、测试脚本、部署脚本、数据库脚本、配置文件和发布包。
2. 版本管理：分支、标签、提交记录和版本号。
3. 变更控制：提交变更请求，影响分析，审批，执行，验证。
4. 配置审计：检查代码、配置、脚本、依赖和部署清单是否一致。
5. 状态报告：记录每个配置项的版本、负责人、状态和变更历史。
6. 发布管理：建立发布基线，执行发布检查和回滚预案。

实验答题模板：

我们将需求文档、设计文档、源代码、测试用例、数据库脚本和部署配置纳入 Git 管理；主分支保持可运行，功能在独立分支开发；合并前通过 Pull Request 审查；重要版本打 Tag；需求或接口变化通过 Issue 记录原因、影响范围和处理结果；发布前核对配置和版本并执行冒烟测试。

题 7：示例：结合实验说明一个项目的质量保障包括哪些活动

答案：

标准结构：

1. 需求质量保障：需求评审、冲突检查、可验证性检查。
2. 设计质量保障：架构评审、接口评审、职责分配检查。
3. 代码质量保障：编码规范、代码审查、静态分析、重构。

4. 测试质量保障：单元测试、集成测试、系统测试、回归测试。
5. 配置质量保障：版本管理、变更控制、配置审计、状态报告。
6. 发布质量保障：发布基线、部署清单、冒烟测试、回滚计划。

答题句：质量保障不是只做测试，而是通过评审、测试、度量、配置管理和过程审计共同降低缺陷进入后续阶段的风险。

第五章：需求工程

题 8：名词解释：需求

答案：

需求是利益相关者为解决问题或达到目标而要求系统具备的能力、行为、约束或质量属性。需求既包括系统要做什么，也包括系统必须满足的性能、安全、可靠性、接口、数据和业务规则约束。

题 9：区分需求的三个层次，并给出实例

答案：

三个层次：

层次	回答的问题	校园卡充值例子
业务需求	为什么建设系统	提高校园卡充值自助化比例，降低人工窗口压力。
用户需求	用户要完成什么任务	学生希望在线给校园卡充值并查看充值结果。
系统级需求	系统在条件下必须做什么	当支付平台返回支付成功结果时，系统应更新校园卡余额并生成充值记录。

判定方法：看需求回答的是“为什么建系统”“用户想完成什么任务”还是“系统必须提供什么可验证行为”。

题 10：掌握需求类型，并对实例判定类型

答案：

常见类型：

- 功能需求：系统要做什么，如“系统应支持用户查询余额”。
- 性能需求：速度、容量、吞吐量、响应时间，如“查询响应时间不超过 2 秒”。
- 质量属性：安全性、可靠性、可维护性、易用性、可用性。
- 对外接口需求：与外部系统、设备、用户界面的交互方式，如“通过 HTTPS 调用支付平台接口”。

- 约束：技术栈、法规、部署环境、开发过程限制。
- 数据需求：数据字段、格式、关系、完整性规则和保留期限。

判定句式：

该需求属于功能需求，因为它描述系统在某个输入或事件下必须执行的业务行为。该需求属于数据需求，因为它规定了字段格式、取值范围或持久化内容。该需求属于对外接口需求，因为它规定了本系统与外部系统的调用方式、数据格式或异常处理。

第六章：需求分析建模

题 11：给定描述时，如何建立用例图、分析类图、系统顺序图和状态图？

答案：

建模顺序：

1. 用例图：识别参与者、系统边界、用户目标、include 和 extend 关系。
2. 分析类图：从需求文本中提取领域名词，保留系统需要长期记录或参与业务规则的概念；只写属性和关联，不写方法。
3. 系统顺序图：把系统看成黑盒，画参与者与系统之间的事件、返回和异常流程，不画内部对象。
4. 状态图：选择生命周期明显的对象，如订单、预约、借阅、校园卡；列出状态、事件、条件和动作。

检查点：边界清楚、粒度适中、概念类不混入界面控件或数据库表、顺序图不画内部实现。

题 12：图书馆系统：建立用例模型并说明建模过程

答案：

建模过程：

1. 识别参与者：借书人、管理员、图书馆管理者、外部大学数据库。借书人有教职工借书人、研究生借书人、本科生借书人三个特化。
2. 识别系统边界：简化大学图书馆系统。
3. 提取用户目标：检索书名、查询可借状态、借书、还书、预约、跟踪新书、打印标题表、检查逾期未还、下载借书人信息。
4. 组织关系：借书人参与检索、借书、预约；管理员参与借书、还书、跟踪新书；管理者参与打印报表和检查逾期；系统访问外部大学数据库下载借书人信息。

文字版用例模型：

参与者	用例
借书人	检索书名、查询图书可借状态、预约图书、办理借书。

管理员	办理借书、办理还书、跟踪新书到达。
图书馆管理者	打印图书标题表、在线检查过期未还图书。
外部大学数据库	提供借书人信息下载服务。

include 和 extend 的写法：办理借书包含“登记借出记录”和“关联图书副本”；当所有副本已借出时，“预约图书”作为条件性扩展发生。

题 13：ATM 系统：建立领域类图，即分析类图

答案：

领域概念和属性：

概念类	属性或说明
ATM	设备编号、位置、状态。
显示屏幕	显示内容。
输入键盘	数字键、特殊符号键。
银行卡读卡器	读卡状态。
存款插槽	接收存款。
收据打印机	打印收据。
客户	客户编号、姓名、安全级别。
银行卡	卡号、状态。
账户	账号、余额、账户状态。
事务	事务编号、时间、类型、金额、结果。
存款事务	存款金额。
取款事务	取款金额。
余额查询事务	查询时间。
安全系统	PIN 验证规则、安全级别分配。
账户系统接口	账户更新、余额查询。

关系：

- ATM 由显示屏幕、输入键盘、读卡器、存款插槽、收据打印机组成。
- 客户持有银行卡，银行卡关联一个或多个账户。
- 客户通过 ATM 发起事务，事务分为存款、取款、余额查询。
- 每次事务执行前调用安全系统验证 PIN。
- 存款、取款和余额查询通过账户系统接口访问或更新账户。

分析类图只表示问题域概念、属性和关联，不写 Controller、Service、Repository 或数据库表。

第七章：需求规格说明与测试

题 14：为什么需要需求规格说明？结合实验说明

答案：

需求规格说明 SRS 的作用：

1. 建立共同理解：让用户、开发、测试和管理者对系统能力达成一致。
2. 支持设计和实现：为架构、接口、数据和测试提供输入。
3. 支持验证：每条需求都应可测试、可追踪。
4. 控制变更：变更发生时能分析影响范围。
5. 形成合同或验收依据：明确系统交付边界。

实验答题模板：

在实验项目中，SRS 记录了功能需求、数据需求、接口需求、非功能需求和验收标准。设计阶段依据 SRS 划分模块和接口；测试阶段依据 SRS 设计功能测试用例；需求变更时根据编号追踪到设计、代码和测试，避免遗漏。

题 15：对给定需求规格说明片段，如何判定并修正错误？

答案：

检查维度：

- 歧义：出现“尽快”“方便”“大量”“正常情况”等不可验证词。
- 不完整：缺少触发条件、输入、输出、异常流程或边界值。
- 不一致：同一术语、状态或业务规则前后冲突。
- 不可验证：没有明确阈值、条件和期望结果。
- 混入设计实现：把数据库表、按钮布局、类名等实现细节写成需求。
- 多需求混写：一个条目同时描述多个行为。

修正模板：

当 < 触发条件 > 发生，且 < 前置条件 > 成立时，系统应 < 可观察响应 >；若 < 异常条件 > 发生，系统应 < 异常响应 >。

例子：把“系统要很快完成查询”改为“当用户提交余额查询请求时，系统应在 2 秒内返回当前余额或失败原因，不包含外部网络故障时间”。

题 16：对给定需求示例，如何设计功能测试用例？

答案：

功能测试基于规格，不看内部代码。设计方法：

1. 等价类划分：有效输入类和无效输入类。
2. 边界值分析：最小值、最大值、刚好越界值。
3. 决策表：多个条件组合导致不同结果。
4. 状态转换：对象在不同状态下接受事件产生不同响应。
5. 场景测试：覆盖主成功流程和关键异常流程。

测试用例表：

编号	输入/前置条件	覆盖点	预期结果
TC-01	正常用户、有效金额、支付成功	主成功流程	更新余额，生成成功记录。
TC-02	金额为 0	无效等价类	拒绝请求并提示金额非法。
TC-03	校园卡已挂失	状态异常	拒绝充值，不更新余额。
TC-04	支付平台返回失败	外部接口异常	记录失败原因并提示用户。

第八章：软件设计基础

题 17：名词解释：软件设计

答案：

软件设计是把需求转换为可实现的软件解决方案的过程。它确定系统的体系结构、模块划分、接口、数据结构、类和对象协作、算法、异常处理和部署约束。设计回答“如何实现需求”。

题 18：软件设计的核心思想是什么？

答案：

核心思想是管理复杂度和变化。主要手段：

- 分解：把复杂系统拆成模块、层、组件和类。
- 抽象：保留关键概念，隐藏不必要细节。
- 信息隐藏：隐藏重要设计决策和变化点。
- 模块化：通过高内聚、低耦合降低修改传播。
- 接口化：依赖稳定抽象而不是具体实现。

题 19：软件工程设计有哪三个层次？各层主要思想是什么？

答案：

三个层次：

层次	主要思想
体系结构设计	确定系统整体结构、分层、组件、连接件、部署节点和技术约束。
详细设计	确定类、接口、职责分配、对象协作、控制风格和方法级结构。
代码设计	关注变量、语句、控制结构、异常处理、命名、布局、可读性和可维护性。

第九、十章：体系结构

题 20：体系结构的概念

答案：

软件体系结构是系统的高层结构，包含组件、连接件和配置。组件承担计算、存储或业务职责；连接件定义组件之间如何交互，如方法调用、REST API、消息队列；配置说明组件和连接件如何组合成系统。

答题句：体系结构决策影响系统的可维护性、性能、安全性、可扩展性和部署方式，一旦落地修改成本高。

题 21：体系结构风格的优缺点

答案：

常用风格：

风格	优点	缺点
分层风格	职责清晰，便于维护、测试和替换下层实现。	层次过多会增加调用开销，错误依赖会破坏结构。
MVC	分离模型、视图和控制，便于界面变化和业务复用。	控制器容易膨胀，前后端职责边界需要明确。
管道-过滤器 客户端-服务器	适合数据流处理，过滤器可复用。 部署边界清楚，客户端和服务端职责分离。	不适合强交互和共享状态复杂的系统。 网络延迟、服务端瓶颈和安全性问题需要控制。
微服务	服务独立部署和扩展，适合大型组织。	分布式事务、监控、运维和接口治理复杂。

题 22：体系结构设计过程

答案：

作答步骤：

1. 分析需求和质量属性。
2. 识别关键场景和架构风险。
3. 选择体系结构风格。
4. 划分逻辑层、组件和包。
5. 设计层间接口和依赖方向。
6. 设计物理部署和通信方式。
7. 用核心用例、集成测试和评审验证架构。

题 23：包的原则

答案：

包用于组织类和接口，原则包括：

- 单一职责：同一包内元素围绕同一职责。
- 稳定依赖：不稳定包依赖稳定抽象，避免稳定包依赖易变实现。
- 无环依赖：包之间不形成循环依赖。
- 依赖接口：上层依赖下层接口而不是实现类。
- 按层和领域组织：Controller、Service、Repository、Domain 等职责边界清楚。

题 24：体系结构构件之间接口的定义

答案：

接口定义组件之间的服务契约，至少说明：

- 接口所在包名和接口名。
- 操作名、参数、返回值和异常。
- 前置条件、后置条件和错误处理。
- 调用方向和依赖方向。

分层系统中，Controller 依赖 Service 接口，ServiceImpl 依赖 Repository 接口，Repository 不处理业务规则。

题 25：体系结构开发集成测试用例：Stub 和 Driver

答案：

集成测试检查模块协作和接口契约。

- Stub: 被测模块调用的下层替身。上层已完成、下层未完成时, 用 Stub 模拟下层返回。
- Driver: 调用被测模块的上层替身。下层已完成、上层未完成时, 用 Driver 触发被测模块。

例子: 测试 Service 时, Repository 未完成, 使用 Repository Stub 返回固定数据; 测试 Repository 时, Controller 未完成, 使用 Driver 调用 Repository 方法并检查数据库结果。

题 26: 薪酬管理系统 Web 分层物理包图如何作答?

答案:

物理包图必须体现客户端、网络通信、服务器端分层和功能模块。

文字版结构:

- 客户端浏览器: html、css、js。
- 网络通信: HTTP, 基于 REST 风格 API。
- 服务器表示入口: controller 包, 包含员工、薪资和社会保险录入等 Controller。
- 逻辑层: service 包和 service.impl 包, 如员工管理、薪资计算、社会保险录入。
- 数据层: repository 包和 repository.impl 包。
- 领域对象: domain 包, 如 Employee、SalaryRecord、SocialInsuranceRecord。
- 数据库: 员工表、薪资表、社会保险表。

得分点: 浏览器在客户端, 逻辑层和数据层在服务器端; 客户端与服务器通过 HTTP/REST 通信; 服务器内部体现 Controller、Service、Repository 和数据库依赖。

题 27: “社会保险录入”页面的展示层、逻辑层、数据层接口声明

答案:

展示层与逻辑层之间的接口放在 payroll.service 包:

```
package payroll.service;

public interface SocialInsuranceService {
    SocialInsuranceResult createRecord(SocialInsuranceCommand command);
    SocialInsuranceRecordDTO findById(EmployeeId String employeeId);
}
```

逻辑层与数据层之间的接口放在 payroll.repository 包:

```
package payroll.repository;

public interface SocialInsuranceRepository {
    void save(SocialInsuranceRecord record);
    SocialInsuranceRecord findById(EmployeeId String employeeId);
}
```

如需员工信息校验，再写员工仓储接口：

```
package payroll.repository;

public interface EmployeeRepository {
    Employee findById(String employeeId);
}
```

解释：Controller 只调用 `SocialInsuranceService`；ServiceImpl 校验员工和保险录入规则后调用 Repository；Repository 只负责数据访问，不写业务规则。

第十一章：人机交互

题 28：名词解释：可用性

答案：

可用性表示特定用户在特定使用环境下，使用系统完成特定任务时达到有效性、效率和满意度的程度。课程答题常写五项属性：易学性、效率、易记性、错误率与恢复、满意度。

题 29：列出至少 5 个界面设计注意事项并解释

答案：

可写以下原则：

1. 一致性：相同功能在不同页面使用一致的术语、位置、颜色和交互方式。
2. 反馈：用户操作后系统立即显示状态、结果或进度。
3. 识别优于回忆：让可选项、历史记录和状态可见，减少用户记忆负担。
4. 错误预防：通过约束输入、默认值、确认框和撤销减少错误。
5. 用户控制和自由：支持取消、返回、撤销和修改。
6. Fitts 定律：高频操作按钮要足够大并靠近当前任务区域。
7. 任务匹配：界面按用户任务组织，而不是按数据库表或技术模块组织。

题 30：分析图中两个界面体现或违反的人机交互设计原则

答案：

左图主要问题：

- 违反识别优于回忆：用户需要通过滚动条逐个查找样式，列表不可见。
- 反馈不足：当前选择和最终效果之间的关系不够直接。

- 效率低：用户需要反复滚动才能比较样式。
- 控件匹配差：横向滚动条不适合浏览大量样式名称。

右图改进点：

- 支持识别优于回忆：左侧列表直接显示多个样式名称。
- 反馈更明确：右侧预览随选择变化，用户能看到结果。
- 效率更高：列表和预览并列，减少来回操作。
- 仍需注意：若列表过长，应提供搜索、分类或更大的预览区域。

题 31：精神模型、差异性、导航、反馈、协作式设计

答案：

答题要点：

- 精神模型：用户对系统如何工作的理解。界面应符合用户的任务习惯和领域概念。
- 差异性：不同用户能力、经验、设备和使用场景不同，界面要兼顾新手、中级用户和专家用户。
- 导航：让用户知道自己在哪里、能去哪、如何返回和下一步是什么。
- 反馈：操作后及时显示状态、结果、错误原因或进度。
- 协作式设计：让用户、设计者、开发者共同参与需求澄清、原型评审和可用性测试。

第十二章：详细设计

题 32：详细设计的出发点是什么？

答案：

详细设计从分析模型、系统顺序图、架构约束和设计因素出发，把“系统必须做什么”转换为“对象如何协作完成”。重点是职责分配、对象协作、控制风格、类和接口设计、方法签名和测试设计。

题 33：职责分配、协作、控制风格分别是什么？

答案：

- 职责分配：决定哪个类负责知道信息、执行操作、创建对象或协调流程。
- 协作：对象之间通过消息调用共同完成用例。
- 控制风格：用例流程由谁主导。

控制风格：

风格	特征	评价
集中式	一个控制对象掌握大量流程。	流程集中但控制器容易臃肿。
委托式	控制器保留主流程,子任务委托给专业对象。	职责清晰,常作为考试推荐结构。
分散式	流程散落在多个对象中。	单个对象小,但控制流难追踪。

题 34：给定分析类图、系统顺序图和设计因素，如何建立设计类图或详细顺序图？

答案：

步骤：

1. 从系统顺序图中找系统事件。
2. 为系统事件选择 Controller。
3. 根据 GRASP 信息专家原则，把业务判断分给拥有数据的领域对象。
4. 为外部依赖设计接口，如 Repository、Gateway、Service。
5. 在设计类图中补充方法、可见性、接口、依赖和实现关系。
6. 在详细顺序图中画 Controller、Service、Domain、Repository 的调用顺序。
7. 用低耦合、高内聚、信息隐藏和可测试性检查设计。

题 35：协作测试中的 MockObject

答案：

MockObject 用来替代外部依赖，使测试集中在当前对象的协作行为上。它能验证：

- 被测对象是否调用了正确依赖。
- 调用参数是否正确。
- 外部依赖返回成功或失败时，被测对象是否正确处理。
- 不访问真实数据库、支付平台或网络服务，从而让单元测试稳定、快速、可重复。

题 36：示例：根据 CD 推荐用例画设计类图和集中式控制顺序图

答案：

原图给出顾客向系统提交查询请求，系统返回 CD 推荐列表，顾客选择 CD 后查询详细信息并购买 CD。集中式控制写法如下：

- 设计类: RecommendationController 作为控制类; Customer、Request、CDRecommendationList、CD 作为领域类; CDRepository 作为数据访问接口。
- 关键方法:
 - queryRecommendations
 - getDetail
 - purchase
- 集中式顺序: Customer -> Controller -> Repository -> Controller -> Customer。后续“选定 CD”“查询详细信息”“购买 CD”也由 Controller 统一协调。

评价: 集中式控制容易画、流程清楚, 但 Controller 会积累推荐、查询和购买逻辑。改进时把推荐规则委托给 RecommendationService, 把购买规则委托给 OrderService。

第十三章: 耦合、内聚与信息隐藏

题 37: 名词解释: 耦合与内聚

答案:

耦合表示模块之间依赖强度。耦合越低, 一个模块变化越不容易影响其他模块。

内聚表示模块内部职责相关程度。内聚越高, 模块越围绕单一目标组织, 越容易理解、测试和修改。

题 38: 如何说明例子中的耦合程度与内聚, 并给理由?

答案:

耦合判断:

- 内容耦合: 直接访问或修改其他模块内部。
- 公共耦合: 多个模块共享全局数据。
- 控制耦合: 传递控制标志让被调用者选择行为。
- 印记耦合: 传递整个对象或结构, 但只使用其中一部分。
- 数据耦合: 只通过必要参数交互。

内聚判断:

- 偶然内聚: 职责无明确关联。
- 逻辑内聚: 用类型参数选择不同逻辑。
- 时间内聚: 因同一时间执行而放在一起。
- 过程内聚: 按流程顺序放在一起。
- 通信内聚: 操作同一数据集合。

- 功能内聚：只完成一个明确功能。
- 信息内聚：围绕同一数据结构组织多个操作。

答题先给类型，再说明依赖方式或职责关系，再给修改影响。

题 39：信息隐藏的基本思想和两种常见信息隐藏决策

答案：

信息隐藏的基本思想：模块隐藏一个重要设计决策，只暴露稳定接口。信息隐藏不是简单把字段设为 private，而是隐藏容易变化的决策。

常见决策：

1. 数据结构或存储方式：用 Repository 接口隐藏数据库表、文件、缓存或远程服务。
2. 算法或业务规则：用 Strategy、Service 或领域对象隐藏不同规则实现。

判断信息隐藏好坏：外部模块是否依赖内部字段、具体类、算法步骤、数据库结构或第三方接口细节。依赖越少，信息隐藏越好。

第十四章：模块化原则

题 40：Principles from Modularization 九条原则及修正方式

答案：

原则	答题要点
Global Variables Consider Harmful	避免全局变量和共享可变状态；改为参数、对象字段、依赖注入或配置对象。
To be Explicit	依赖、输入、输出和异常要显式表达；不要依赖隐式全局状态。
Do not Repeat	不重复知识和逻辑；重复代码提取方法、类或配置表。
Programming to Interface	面向接口编程，依赖抽象而不是实现。
Law of Demeter	只和直接朋友通信，避免长消息链。
Interface Segregation Principle	接口小而专用，客户端不依赖自己不用的方法。
Liskov Substitution Principle	子类必须能替换父类，不破坏父类语义。
Favor Composition Over Inheritance	优先组合和委托，减少白盒复用风险。
Single Responsibility Principle	一个模块只有一个变化理由。

对给定代码题的作答结构：指出违反的原则，说明风险，给出改写结构，再补充测试。

第十五章：封装、OCP、DIP 与 LSP

题 41：信息隐藏与封装的含义

答案：

信息隐藏关注“隐藏设计决策和变化点”；封装关注“把数据和操作组织在模块或对象内部，并通过接口访问”。封装是实现信息隐藏的重要手段，但有封装语法不代表真正完成信息隐藏；如果暴露内部结构或让外部依赖实现细节，仍然信息隐藏不足。

题 42：OCP 和 DIP

答案：

OCP：对扩展开放，对修改关闭。新增业务规则时通过新增类或配置扩展，不频繁修改稳定核心代码。

DIP：高层模块不依赖低层模块，二者都依赖抽象；抽象不依赖细节，细节依赖抽象。典型写法是 Controller 依赖 Service 接口，ServiceImpl 依赖 Repository 接口。

题 43：替换原则例题：MyStack 继承 Vector 是否合理？Employee 继承 Person 是否合理？

答案：

MyStack extends Vector 不合理。栈只允许压栈、弹栈、取大小和判空，但继承 Vector 后会暴露 insertElementAt、removeElementAt、elementAt 等任意位置操作，外部能绕过栈的后进先出约束。这违反 LSP，也违反封装和组合优于继承。修改方式是组合：

```
public class MyStack {
    private final Vector<Object> elements = new Vector<>();

    public void push(Object element) {
        elements.insertElementAt(element, 0);
    }

    public Object pop() {
        Object result = elements.firstElement();
        elements.removeElementAt(0);
        return result;
    }

    public int size() {
        return elements.size();
    }

    public boolean isEmpty() {
        return elements.isEmpty();
    }
}
```

```
}
```

Employee extends Person 合理。只要业务语义满足“每个 Employee 都是 Person”，且 Employee 不改变 Person 的基本契约，就符合 LSP。Employee 可以增加员工号、部门、薪资等属性，但不能让 `getName()` 的含义变化，也不能增强父类方法的前置条件或削弱后置条件。

第十六章：设计模式

题 44：如何实现可修改性、可扩展性和灵活性？

答案：

方法：

- 找变化点：算法、产品族、创建逻辑、遍历方式、外部接口。
- 抽象稳定接口：让高层代码依赖接口。
- 封装变化：把变化逻辑放入独立实现类。
- 使用组合和委托：运行时替换实现。
- 用测试保护变化：为接口契约和核心场景写测试。

题 45：策略模式

答案：

定义一组算法，把每个算法封装起来，并让它们可以互换，使算法变化独立于使用算法的客户。

适用场景：同一业务存在多种算法或规则，如会员折扣、支付方式、排序算法、积分规则。

结构：Context 依赖 Strategy 接口；每个具体算法实现一个 ConcreteStrategy。新增算法时新增实现类，不修改 Context。

题 46：抽象工厂模式

答案：

提供一个创建一组相关或相互依赖对象的接口，而不指定它们的具体类。

适用场景：需要切换一组产品族，如 MySQL DAO 与 Oracle DAO、Windows UI 控件与 macOS UI 控件。抽象工厂保证同一产品族的一致性。

题 47：单件模式

答案：

单件模式保证一个类只有一个实例，并提供全局访问点。适用于全局唯一且无可变共享风险的对象，如只读配置、日志器入口。

风险：容易形成全局状态、隐式耦合和测试困难。课程答题中要说明边界：不要把数据库连接、业务服务和可变缓存随意做成单件。

题 48：迭代器模式

答案：

迭代器模式在不暴露聚合对象内部表示的前提下，顺序访问其中元素。适用于隐藏集合内部结构，如数组、链表、分页查询、数据库游标。客户端只依赖 Iterator 接口，不关心底层存储。

题 49：给定场景选择设计模式并写代码，给出代码要求用设计模式改写

答案：

答题模板：

1. 找变化点：多种算法用 Strategy；一组相关对象创建用 Abstract Factory；全局唯一对象用 Singleton；隐藏遍历结构用 Iterator。
2. 写出角色：接口、具体实现、上下文或工厂。
3. 说明原则：OCP、DIP、低耦合、封装变化。
4. 写测试：新增实现类后旧流程不变。

策略模式骨架：

```
public interface DiscountStrategy {
    Money discount(Order order);
}

public class VipDiscountStrategy implements DiscountStrategy {
    public Money discount(Order order) {
        return order.total().multiply(0.8);
    }
}

public class OrderService {
    private final DiscountStrategy strategy;

    public OrderService(DiscountStrategy strategy) {
        this.strategy = strategy;
    }
}
```

```
public Money calculatePayable(Order order) {  
    return strategy.discount(order);  
}  
}
```

第十七、十八章：软件构造与代码设计

题 50：构造包含哪些活动？

答案：

构造不只是编码，包含：

- 编码和单元实现。
- 单元测试、调试和代码审查。
- 集成、构建和部署制品生成。
- 静态分析、持续集成和质量门禁。
- 重构、缺陷修复和开发者文档。

题 51：名词解释：重构、测试驱动开发、结对编程

答案：

- 重构：在不改变软件外部行为的前提下改善内部结构。
- 测试驱动开发 TDD：Red、Green、Refactor。先写失败测试，再写最少代码通过测试，最后在测试保护下重构。
- 结对编程：两名开发者共同完成同一任务，一人驾驶编码，一人观察审查，定期交换角色。

题 52：给定代码段，如何发现问题并改进？

答案：

从以下角度分析：

- 命名：变量、方法和类名是否表达意图。
- 简洁性：是否存在长方法、重复代码、过深嵌套。
- 职责：是否一个方法承担校验、计算、持久化和展示多个职责。
- 控制结构：是否能用早返回、卫语句、多态或表驱动简化。
- 数据结构：是否用合适结构替代复杂条件判断。
- 异常处理：是否吞掉异常、返回魔法值或缺少错误恢复。

- 测试：是否能为正常、边界和异常场景编写测试。

题 53：变量使用、语句处理和 How to write unmaintainable code

答案：

变量使用：

- 缩小作用域，减少可变状态。
- 使用有意义命名，避免魔法数字。
- 用常量表达业务阈值。

语句处理：

- 使用卫语句减少嵌套。
- 复杂条件提取为命名方法。
- 避免超长方法和重复分支。

不可维护代码的典型问题：命名无意义、全局变量、重复代码、隐藏副作用、过深嵌套、类型码分支、异常被吞掉、缺少测试。

题 54：防御与错误处理、契约式设计、防御式编程

答案：

契约式设计强调前置条件、后置条件和不变量。调用者满足前置条件，被调用者保证后置条件。

防御式编程强调即使输入不可信也要检查和处理：

- 校验空值、范围、格式和状态。
- 对非法输入给出明确异常或错误结果。
- 外部资源使用超时、重试和降级。
- 不吞异常，记录必要上下文。

题 55：表驱动

答案：

表驱动用数据表替代复杂分支，适合规则稳定、分支很多的场景。数据保存的重点是节省计算、减少重复判断并提高可维护性。

月份天数示例：


```
private static final int[] DAYS = {
    0, 31, 28, 31, 30, 31, 30,
    31, 31, 30, 31, 30, 31
};

public int daysOfMonth(int month) {
    if (month < 1 || month > 12) {
        throw new IllegalArgumentException("month");
    }
    return DAYS[month];
}
```

题 56：单元测试用例设计

答案：

单元测试关注单个类或方法：

- 正常输入。
- 边界输入。
- 非法输入。
- 状态变化。
- 外部依赖用 Mock 替代。

表格格式：编号、前置条件、输入、覆盖点、预期输出。

第十九章：测试

题 57：黑盒测试和白盒测试的常见方法，并比较优缺点

答案：

黑盒测试基于规格，不看内部代码。方法包括等价类划分、边界值分析、决策表、状态转换、场景测试。优点是贴近用户需求；缺点是无法直接保证代码路径覆盖。

白盒测试基于代码结构。方法包括语句覆盖、分支覆盖、条件覆盖、路径覆盖。优点是能发现内部逻辑遗漏；缺点是依赖代码结构，不能替代需求层面的功能验证。

题 58：解释并区别语句覆盖、分支覆盖和路径覆盖

答案：

- 语句覆盖：每条语句至少执行一次。强度最低，不能保证每个判定结果都执行。

- 分支覆盖：每个判定的真分支和假分支至少执行一次。强于语句覆盖。
- 路径覆盖：覆盖程序中的执行路径。强度高，但路径数量随条件组合快速增长，成本最高。

题 59：给定场景，如何选择测试方法并写测试用例？

答案：

选择规则：

- 给出功能需求：写功能测试，用等价类、边界值、决策表和场景测试。
- 给出设计图：写集成测试，关注接口、消息顺序、Stub 和 Driver。
- 给出方法代码：写单元测试，关注白盒覆盖、边界和异常。
- 给出外部依赖：使用 Mockito 隔离数据库、网络或第三方服务。

测试用例格式：

编号	输入/前置条件	覆盖点	预期结果
TC-01	有效输入	正常路径	返回成功结果。
TC-02	最小边界值	边界值	接受或返回规定错误。
TC-03	刚好越界	无效等价类	拒绝并提示原因。
TC-04	依赖服务失败	异常路径	回滚或返回失败结果。

题 60：JUnit 基本使用方法

答案：

JUnit 测试通常包含 Arrange、Act、Assert 三步：

```
@Test
void shouldRejectZeroAmount() {
    RechargeService service = new RechargeService(fakeRepository);

    assertThrows(IllegalArgumentException.class, () -> {
        service.recharge("card-001", BigDecimal.ZERO);
    });
}
```

要点：测试名表达行为；每个测试验证一个重点；使用断言检查返回值、异常或状态；外部依赖用 Fake、Stub 或 Mock 替代。

第二十、二十一章：维护与演化

题 61：如何理解软件维护的重要性？

答案：

软件交付后仍会因需求变化、运行环境变化、缺陷修复、安全漏洞、性能压力和业务扩展而持续修改。维护阶段通常占据软件生命周期的大量成本。可维护性差会导致修改风险高、缺陷回归多、交付速度慢。

题 62：开发可维护软件的方法

答案：

方法：

- 清晰需求和可追踪关系。
- 分层架构、低耦合、高内聚。
- 信息隐藏和稳定接口。
- 编码规范、清晰命名和小方法。
- 自动化测试、持续集成和回归测试。
- 配置管理、版本控制和变更记录。
- 必要文档，包括用户文档和系统文档。

题 63：演化式生命周期模型

答案：

演化式生命周期模型先交付初始版本或原型，再根据用户反馈、环境变化和新需求持续修改。它适合需求不稳定、问题域需要探索的系统。

优点：早反馈，能吸收真实用户体验。缺点：范围控制难，架构规划不足时会退化为不断打补丁。

题 64：用户文档和系统文档

答案：

- 用户文档：面向使用者，说明安装、登录、功能操作、常见问题和错误处理。
- 系统文档：面向开发和维护人员，说明需求、架构、设计、接口、数据库、部署、测试和运维。

用户文档帮助正确使用系统，系统文档帮助理解、维护和演化系统。

题 65：逆向工程和再工程

答案：

逆向工程：从已有代码或系统中恢复设计、需求、架构或业务规则。

再工程：在理解旧系统的基础上重构、迁移、重写或改造，使系统更易维护、适配新环境或满足新需求。

区别：逆向工程侧重理解和恢复；再工程侧重改造和提升。

第二十二、二十三章：生命周期模型与知识体系

题 66：软件生命周期模型

答案：

软件生命周期是软件从提出、开发、运行、维护到退役的完整过程。生命周期模型描述这些活动的组织方式。常见模型包括 Build-and-fix、瀑布模型、增量迭代模型、演化模型、原型模型和螺旋模型。

题 67：解释并比较不同过程模型

答案：

模型	特征	优点	缺点
Build-and-fix	直接编码，边做边改。	启动快。	过程不可控，质量和维护风险高。
瀑布模型	阶段顺序清楚，文档驱动。	适合需求稳定、合同明确项目。	反馈晚，早期错误传递到后期。
增量模型	分批交付可运行功能。	早交付，风险分散。	需要总体架构和增量规划。
迭代模型	多轮重复分析、设计、实现和测试。	持续改进。	迭代控制不当会反复返工。
演化模型	初始版本随反馈持续演化。	适合需求变化。	范围和进度控制难。
原型模型	用原型澄清需求。	帮助用户表达需求。	抛弃式原型不能直接当最终产品。
螺旋模型	风险驱动，每轮解决主要风险。	适合大型高风险项目。	风险分析能力要求高，管理成本高。

题 68：给定场景，判定适用的开发过程模型

答案：

判定规则：

- 需求稳定、合同边界清楚：瀑布模型。

- 核心需求清楚、功能可按优先级分批交付：增量迭代模型。
- 需求变化频繁、需要边用边改：演化模型。
- 用户说不清界面、流程或业务规则：原型模型。
- 大型、高风险、技术和需求不确定性高：螺旋模型。

题 69：软件工程知识体系的知识域

答案：

SWEBOK 覆盖需求、设计、构造、测试、维护、配置管理、工程管理、过程、模型与方法、质量、职业实践、经济学、计算基础、数学基础、工程基础等知识域。

SWEBOK V4 增强关注软件体系结构、软件工程运维、软件安全、AI for SE 和 SE for AI。答题收束句：AI 可以提高需求整理、设计草图、代码生成和审查效率，但工程纪律仍由需求验证、接口约束、测试、配置管理和人工评审保证。

题型和数量

题 70：考试题型和数量如何理解？

答案：

原提纲最后写明数量为 8 到 10 道大题，题型为中文试题，全部是大题。复习时按大题组织答案：定义题写关键词和展开句；建模题写过程和模型元素；代码题写违反原则、影响、修改和测试；案例题写问题、原因、改进和验证。